

GOVERNMENT WELFARE ADVISORY SYSTEM



A PROJECT REPORT

Submitted by

KARTHI C	731122104024
SANJAY KUMAR S	731122104044
SHANMUGA RAJA M	731122104047

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

GOVERNMENT COLLEGE OF ENGINEERING,

ERODE – 638316

ANNA UNIVERSITY: CHENNAI 600025

APRIL 2026

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report “**GOVERNMENT WELFARE ADVISORY SYSTEM**” is the bonafide work of “**KARTHI C (731122104024), SANJAY KUMAR S (731122104044), SHANMUGA RAJA M (731122104047),**” who carried out the project under my supervision.

SIGNATURE

SIGNATURE

Mrs. S. VIJAYALAKSHMI M.E.,

Dr. A. KAVITHA, M.E., Ph.D.,

SUPERVISOR

HEAD OF THE DEPARTMENT

Department of CSE

Department of CSE

GOVERNMENT COLLEGE OF

GOVERNMENT COLLEGE OF

ENGINEERING, ERODE – 638316

ENGINEERING, ERODE - 638316

Submitted the university examination held on _____ at Government College of Engineering, Erode.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our whole-hearted thanks to our Principal **Dr. A. SARADHA M.E., Ph.D.**, Government College of Engineering, Erode for his constant encouragement and invaluable support during the course of this project.

We owe our sincere thanks to **Dr. A. KAVITHA, M.E., Ph.D., Professor and Head of the Department**, Department of Computer Science and Engineering, Government College of Engineering, Erode, for providing every essential facility required for completing this project.

We sincerely thank our guide **Mrs. S. VIJAYALAKSHMI M.E., Assistant Professor**, Department of Computer Science and Engineering, Government College of Engineering, Erode, for her valuable help and guidance throughout the project.

We sincerely thank our Coordinator **Dr. R. KALAIVANI, M.E., Ph.D., Assistant Professor**, Department of Computer Science and Engineering, Government College of Engineering, Erode, for her valuable help and guidance throughout the project.

We wish to express our sincere thanks to all the staff members of the Department of Computer Science and Engineering for their valuable suggestions and guidance rendered to us throughout the project.

Above all, we are grateful to all our classmates and friends for their friendly cooperation and their exhilarating company throughout the course of this endeavour.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF TABLES	iii
	LIST OF ABBREVIATIONS	iv
1	INTRODUCTION	
	1.1 PROBLEM STATEMENT	2
	1.2 MOTIVATION	3
	1.3 PROJECT SCOPE	5
2	LITERATURE REVIEW	
	2.1 EXISTING SYSTEM	9
3	SYSTEM SPECIFICATIONS	
	3.1 HARDWARE REQUIREMENTS	11
	3.2 SOFTWARE REQUIREMENTS	11
4	WELFARE ADVISORY SYSTEM	
	4.1 PROPOSED SYSTEM ARCHITECTURE	12
	4.2 MODULE DESCRIPTION	14

	4.3 DATABASE DESIGN	24
5	IMPLEMENTATION AND EXPERIMENTATION RESULT	
	5.1 IMPLEMENTATION	30
	5.2 EXPERIMENTATION RESULT	
	5.2.1 User Dashboard	35
	5.2.2 Eligibility Check Form	37
	5.2.3 Eligibility Result Page	39
	5.2.4 Track Applications Page	41
	5.2.5 Officer Review Dashboard	42
	5.2.6 Admin Applications Management	43
	5.2.7 System Administration Dashboard	45
	5.2.8 Officer Dashboard	48
6	CONCLUSION	
	6.1 Conclusion	51
	6.2 Future Enhancements	52
	APPENDIX	
	A1 Source Code	53

A2 Github Link	58
A3 API Endpoints	58
REFERENCES	59

ABSTRACT

The Government Welfare Advisory System is designed to make it easier for citizens to find and access government welfare schemes that suit their eligibility. In reality, many people miss out on these benefits because they are not aware of available schemes, find the eligibility rules confusing, or struggle to navigate different government websites. To overcome these issues, this system brings everything together into a single, easy-to-use platform where users can quickly check their eligibility and get suitable scheme recommendations without much effort.

The system follows a full-stack development approach, using a React-based frontend to provide a smooth and interactive user experience, and a Spring Boot backend to handle the core logic and data processing. A MySQL database is used to store and organize details related to users, schemes, applications, and eligibility records. It supports different types of users such as citizens, government officers, and administrators, each having their own set of responsibilities. Citizens can browse schemes, check their eligibility, apply for schemes, and track their application status. Officers are responsible for verifying applications and making decisions, while administrators manage schemes, users, and overall system activities. In addition, a machine learning model is included to calculate an eligibility score using factors like age, gender, income, occupation, and social category.

The system offers a more structured and convenient way for people to access government welfare schemes by making the entire process simpler and more efficient. It saves time for users by reducing the need to search across multiple platforms and helps authorities handle applications in a more organized manner. By using modern web technologies along with predictive analysis, the system creates a smooth, transparent, and user-friendly experience for distributing welfare benefits.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Architecture Diagram	13
5.1	Implementation Diagram	31
5.2.1	User Dashboard	36
5.2.2	Eligibility Check Form	38
5.2.3	Eligibility Result Page	40
5.2.4	Track Applications Page	41
5.2.5	Officer Review Dashboard	43
5.2.6	Admin Applications Management	45
5.2.7	System Administration Dashboard	47
5.2.8	Officer Dashboard	49

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Hardware Requirements	11
3.2	Software Requirements	11
4.1	Users Table	27
4.2	Schemes Table	27
4.3	Applications Table	28
4.4	Eligibility Result Table	28
4.5	Officer Table	29
A.1	API Endpoints	58

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
API	Application Programming Interface
BE	Bachelor of Engineering
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
ML	Machine Learning
MVC	Model View Controller
MySQL	My Structured Query Language
npm	Node Package Manager
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

Government welfare schemes play a crucial role in improving the socio-economic conditions of citizens by providing financial assistance, healthcare support, education benefits, housing facilities, and other essential services. These schemes are introduced by governments to support different sections of society such as low-income families, farmers, students, senior citizens, and minority communities. The primary objective of these programs is to reduce poverty, promote equality, and ensure that essential resources reach the people who need them the most. However, despite the availability of many welfare schemes, a large number of eligible citizens are unable to take advantage of them.

One of the major reasons for this issue is the lack of awareness and accessibility. Information about government schemes is often spread across multiple websites and portals, making it difficult for citizens to find relevant programs. Many people are also unaware of the eligibility criteria or the application procedures required to receive these benefits. As a result, individuals who qualify for these schemes may never apply for them simply because they do not know they exist or do not understand how to access them. This problem becomes even more significant for users with limited digital literacy.

The **Government Welfare Advisory System** has been developed as a centralized digital platform that helps citizens easily discover and apply for welfare schemes. The main aim of this system is to simplify the process of identifying suitable schemes based on a user's personal and socio-economic details. By entering information such as age, income level, occupation, gender, and category, users can receive recommendations for government schemes that match their eligibility profile. This reduces the need for manual searching across

multiple websites and improves the chances of citizens accessing the benefits they deserve.

The system is designed using a modern full-stack architecture that combines both frontend and backend technologies. The user interface is developed using React and TypeScript, providing a responsive and user-friendly experience. The backend is implemented using Spring Boot, which handles business logic, user authentication, and communication with the database. A MySQL database is used to store information related to users, welfare schemes, applications, and eligibility history.

The system integrates a machine learning model that helps predict a user's eligibility score based on their provided information. This predictive approach improves the accuracy of scheme recommendations and allows users to quickly identify the most relevant welfare programs.

The Government Welfare Advisory System aims to bridge the gap between citizens and government welfare programs by providing a simple, accessible, and efficient platform for discovering and applying for government benefits.

1.1 Problem Statement

Government welfare schemes are created to support citizens in areas such as financial assistance, education, healthcare, and social development. However, accessing these schemes is often not simple for many people. Information about different schemes is usually spread across multiple government websites and portals, making it difficult for citizens to stay aware and updated. In addition, understanding the eligibility criteria becomes confusing, as each scheme has different conditions based on factors like income, age, occupation, and social category. Most existing platforms provide only basic information, requiring users to manually search and analyze whether they qualify for a scheme.

Due to these challenges, many citizens either miss out on useful benefits or apply for schemes without clearly knowing their eligibility. This results in a large number of invalid or unnecessary applications. At the same time, the lack of personalized recommendations makes it harder for users to identify schemes that truly match their needs. Government officers are then required to manually review a high volume of applications, which increases their workload and slows down the approval process. Managing application data, eligibility checks, and scheme details across different systems also creates inefficiencies, while citizens often face difficulty in tracking their application status, reducing transparency in the overall process.

These issues highlight the gap in the current system and the need for a more efficient solution. There is a strong requirement for a centralized and intelligent platform that can guide citizens, simplify the process of finding and applying for schemes, and provide personalized recommendations based on user details. Such a system can also support government authorities by reducing manual effort, improving accuracy, and ensuring a smoother and more transparent welfare distribution process.

1.2 Motivation of Welfare System

Government welfare schemes are created to support citizens by providing financial assistance, education benefits, healthcare support, and other essential services. These schemes are especially important for economically weaker sections of society who rely on government assistance to improve their quality of life. However, in many cases, eligible individuals do not benefit from these programs because they are unaware of the available schemes or find it difficult to understand the eligibility criteria and application procedures. This gap between

the availability of welfare programs and citizens' access to them highlights the need for a more organized and user-friendly system.

With the increasing use of digital technology, it is possible to simplify the way citizens interact with government services. A centralized digital platform can help users easily explore welfare schemes, check their eligibility, and apply for suitable programs without navigating through multiple websites. Integrating modern technologies such as web applications, databases, and machine learning can further improve the efficiency of this process by providing personalized recommendations based on user information. This motivates the development of the **Government Welfare Advisory System**, which aims to bridge the gap between citizens and welfare schemes while making the overall process more transparent and accessible.

Motivating Factors:

Many citizens are not aware of the government welfare schemes they are eligible for, mainly because the information is spread across multiple platforms and is not easily accessible in one place. This makes it difficult for people to stay informed and take advantage of available benefits. In addition, the eligibility criteria for different schemes are often complex, and users find it hard to understand whether they qualify without proper guidance.

As a result, citizens spend a lot of time manually searching for schemes and trying to interpret their requirements. At the same time, government officers receive a large number of applications that do not meet eligibility conditions, which increases their workload and delays the approval process. The lack of a proper recommendation system also makes it difficult for users to identify schemes that best match their needs.

To address these challenges, there is a need for a centralized and intelligent platform that can recommend suitable schemes based on user data and simplify

the process of checking eligibility and applying. Such a system can also improve transparency by allowing citizens to easily track their application status. Overall, a smart digital solution can make welfare distribution more efficient and ensure that government benefits reach the people who truly need them.

1.3 Project Scope

The Government Welfare Advisory System is developed as a centralized platform that helps citizens easily discover government welfare schemes and understand whether they are eligible. Many people miss out on these benefits because they are not aware of available schemes or find it difficult to understand the eligibility requirements. This system simplifies the entire process by allowing users to enter their personal and socio-economic details and receive suitable scheme recommendations.

The system also supports different types of users such as citizens, government officers, and administrators to ensure smooth and efficient management. Citizens can explore available schemes, apply for benefits, and track the status of their applications. Officers are responsible for reviewing applications and making decisions, while administrators manage scheme details and user access within the system.

The platform includes essential features such as user registration, login, and profile management, which allow citizens to securely store and update their personal information. It also performs eligibility checks based on key factors like income, age, and occupation, making it easier for users to understand which schemes they qualify for without confusion.

The system uses a machine learning model to recommend suitable welfare schemes based on user data. Citizens can apply for schemes directly through the

platform and monitor their application progress. Officers are provided with a dedicated dashboard to review and manage applications efficiently, while administrators handle overall system operations including scheme management and user control.

All application details and eligibility history are stored in a structured database, ensuring proper record management and easy access to information. By combining modern web technologies with intelligent recommendations, the system improves transparency, accessibility, and efficiency in the distribution and management of government welfare schemes.

CHAPTER 2

LITERATURE REVIEW

Digital Platforms for Government Welfare Services

Government welfare schemes are designed to support citizens by providing financial assistance, healthcare benefits, education support, and social security services. However, several studies indicate that many eligible citizens fail to benefit from these programs due to lack of awareness and difficulty accessing information. Traditional government portals often contain large amounts of scheme-related information, but the data is usually distributed across multiple websites. This fragmented structure makes it difficult for citizens to identify suitable schemes quickly.

Researchers suggest that centralized digital platforms can significantly improve the accessibility of welfare schemes. By integrating scheme details, eligibility conditions, and application processes into a single platform, users can easily explore available programs and understand the requirements. Such platforms not only improve accessibility but also simplify the process of applying for government benefits.

Welfare Scheme Recommendation Systems

Recent research has focused on the development of intelligent recommendation systems that help citizens identify government schemes that match their personal and socio-economic conditions. Sharma et al. [1] proposed a system called **GovGenius**, which provides a web-based platform for recommending government welfare schemes to citizens. The system collects user information such as age, income, occupation, and social category and compares it with scheme eligibility criteria to generate recommendations.

The research highlights that many citizens remain unaware of welfare schemes due to the absence of structured advisory systems. By providing personalized recommendations, the system helps users easily identify relevant programs without manually searching through multiple government websites. This approach improves accessibility and encourages greater participation in government welfare initiatives.

Artificial Intelligence in Welfare Management Systems

Artificial intelligence has become an important tool in improving public service delivery. Singh et al. [2] proposed an **AI-driven welfare scheme assistance system** that uses machine learning techniques to analyze user profiles and recommend suitable government schemes. The system processes demographic attributes such as age, education level, income, and occupation to determine eligibility for different welfare programs.

The study highlights that traditional manual approaches for identifying eligible beneficiaries can be inefficient and time-consuming. AI-based systems can automate the process of analyzing user data and matching it with scheme criteria. This reduces the time required to identify relevant schemes and improves the efficiency of government service delivery.

Machine Learning for Beneficiary Identification

Another area of research focuses on the use of machine learning algorithms to identify eligible beneficiaries for welfare schemes. Reddy et al. [3] proposed a **smart beneficiary scheme mapping system** that analyzes socio-economic data such as income level, employment status, and demographic characteristics to predict eligibility for welfare programs.

The study demonstrates that machine learning algorithms such as Decision Trees and Random Forest can be used to classify individuals based on their eligibility for government benefits. By using predictive models, governments can

improve the accuracy of beneficiary identification and reduce errors in the welfare distribution process. This approach also helps reduce administrative workload by automatically filtering eligible applicants.

From the reviewed literature, it is evident that digital platforms combined with machine learning techniques can significantly improve the accessibility and efficiency of welfare scheme management systems. Intelligent recommendation systems help citizens identify suitable welfare programs based on their personal information, while predictive models assist governments in accurately identifying eligible beneficiaries.

The proposed **Government Welfare Advisory System** builds upon these concepts by integrating a centralized web platform with a machine learning-based eligibility prediction model. The system allows citizens to enter their personal and socio-economic information, automatically evaluates eligibility conditions, and recommends relevant welfare schemes. This approach aims to simplify scheme discovery, improve transparency, and ensure that welfare benefits reach the intended beneficiaries more efficiently.

2.1 Existing System

Government Welfare Scheme Portals

Currently, government welfare schemes are mainly accessed through official government portals and websites. These platforms provide information about various schemes, including eligibility criteria, benefits, and application procedures. Citizens who wish to apply for welfare programs must visit these websites, search for relevant schemes, and manually read the details to determine whether they qualify. Some portals also allow users to submit applications online and track the status of their requests.

Manual Scheme Identification Process

Although these portals provide access to important information, they mostly function as information repositories rather than intelligent advisory systems. Users are often presented with long lists of schemes without personalized recommendations based on their profile. As a result, citizens must spend considerable time analyzing scheme requirements and comparing them with their own details to determine eligibility. This process can be confusing, especially for individuals with limited technical knowledge or digital experience.

Administrative Verification Process

In many existing systems, government authorities must manually review and verify the applications submitted by citizens. Officers check each application to ensure that the applicant meets the eligibility criteria of the selected scheme. This manual verification process increases administrative workload and may delay the approval or rejection of applications. Therefore, while current systems provide basic access to welfare scheme information, they do not fully address the challenges related to scheme discovery, eligibility analysis, and efficient application management.

Limitations of Existing System

- Welfare scheme information is spread across multiple government websites and portals.
- Citizens must manually search for schemes and check eligibility requirements.
- Existing systems do not provide personalized scheme recommendations.
- Understanding eligibility criteria can be difficult for many users.
- The process of identifying suitable schemes is time-consuming.
- Many eligible citizens remain unaware of the schemes they qualify for.
- Government officers must manually verify a large number of applications.

CHAPTER 3

SYSTEM SPECIFICATIONS

The Government Welfare Advisory System requires minimal hardware as it is a web-based platform. The software requirements include the tools, languages, and frameworks needed to build and run the system.

3.1 Hardware Requirements

Table 3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 20 GB free disk space
Display	Monitor with at least 1366 × 768 resolution
Network	Stable internet connection

3.2 Software Requirements

Table 3.2 Software Requirements

Category	Software / Technology
Operating System	Windows / Linux / macOS
Frontend	React.js, TypeScript, Tailwind CSS
Backend	Java, Maven, Spring Boot
Database	MySQL
Machine Learning	Python, Scikit-learn
Development Tools	VS Code / IntelliJ IDEA, Git, Postman

CHAPTER 4

WELFARE ADVISORY SYSTEM

4.1 Proposed System Architecture

The proposed system is an **Welfare Advisory System** designed to provide personalized recommendations of government welfare schemes based on user-specific socio-economic data. The primary goal of this system is to simplify access to government benefits and ensure that eligible citizens can easily identify schemes that are most relevant to their needs.

In the existing system, users are required to manually browse multiple government websites, understand complex eligibility criteria, and identify applicable schemes. This process is not only time-consuming but also leads to a lack of awareness, especially among rural and less digitally literate populations. As a result, many deserving individuals fail to benefit from available government initiatives.

To overcome these limitations, the proposed system introduces an automated and intelligent approach. The system collects user inputs such as age, gender, income level, occupation, category (such as General, OBC, SC/ST), and geographic location. These inputs are processed using a machine learning model, which evaluates the user's eligibility and generates a score indicating how suitable a particular scheme is for the user.

Based on this eligibility score, the system filters and ranks government schemes from a centralized database and presents the most relevant options to the user. This eliminates the need for manual searching and ensures that users receive accurate and personalized recommendations.

Fig No 4.1 This figure shows Government Welfare Advisory System Architecture Diagram

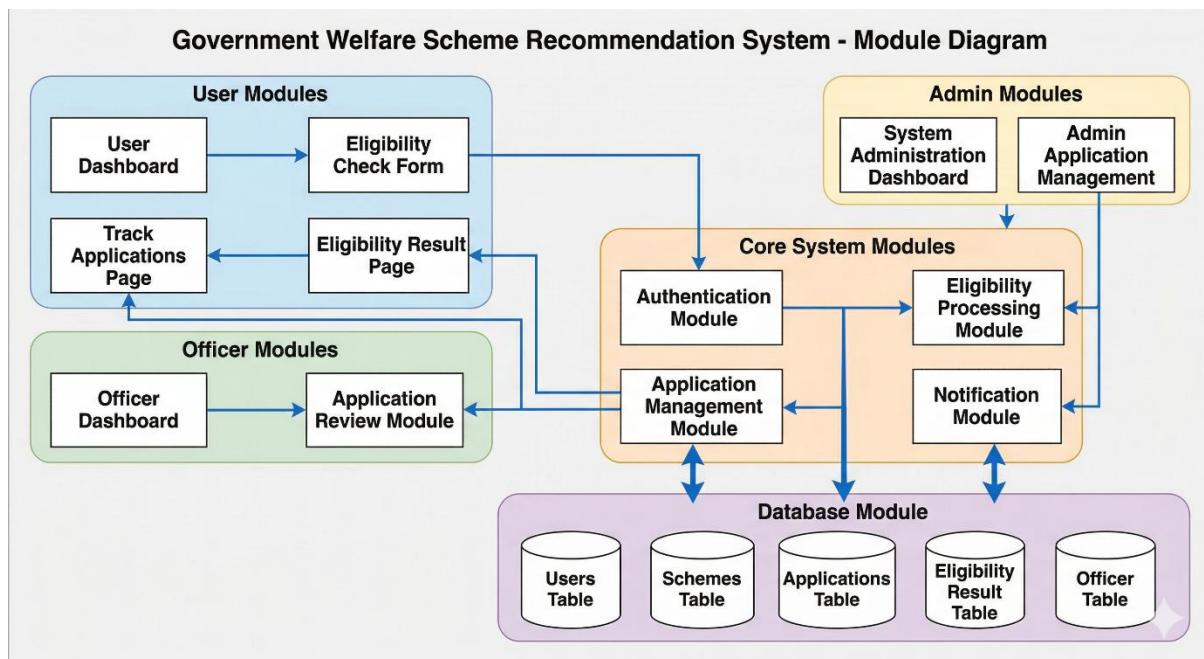


Fig No 4.1 Architecture Diagram

The system is designed using a modular architecture, where different components such as the user interface, backend processing, database, and machine learning model work together seamlessly. This modular approach ensures scalability, flexibility, and ease of maintenance.

The proposed system enhances accessibility, improves efficiency, and promotes awareness of government welfare schemes by leveraging modern technologies such as web development frameworks and machine learning algorithms.

The system design follows a three-tier architecture, which separates the application into three distinct layers: the presentation layer, the application layer, and the data layer. This architectural pattern improves system organization, enhances scalability, and allows independent development and maintenance of each layer.

The presentation layer is responsible for user interaction and includes all frontend components such as forms, dashboards, and display pages. The application layer acts as the core processing unit where business logic is implemented, and communication between different components is managed. The data layer consists of the database and machine learning model, which handle data storage and intelligent prediction respectively.

The system is designed to ensure smooth communication between these layers using RESTful APIs. The use of APIs ensures that the frontend and backend are loosely coupled, making the system flexible and easy to upgrade.

4.2 Module Description

The Welfare Advisory System is divided into multiple functional modules, each designed to handle a specific responsibility within the system. This modular design helps in improving system organization, simplifies development, and allows independent maintenance and scalability of each component. Each module interacts with others through structured data exchange, ensuring smooth workflow and efficient processing of user requests.

User Dashboard Module

The User Dashboard serves as the primary interface for users after successful login into the system. It provides a personalized and organized view of all user-related information, including application summaries, eligibility status, and important notifications. This centralized dashboard ensures that users can quickly access key features without navigating through multiple pages, thereby improving usability and efficiency.

The dashboard displays a summary of the user's activities such as the total number of applications submitted, approved, pending, or rejected. It may also

highlight newly available government schemes that match the user's profile. Notifications regarding application updates, deadlines, or system announcements are prominently shown to keep users informed at all times.

The User Dashboard acts as a navigation hub by providing quick links to other modules like Eligibility Check, Track Applications, and Profile Management. By offering a user-friendly layout and real-time updates, this module enhances user engagement and ensures a seamless experience while interacting with the system.

Sample JSON Request

```
{  
  "userId": "U101",  
  "name": "Karthi",  
  "totalApplications": 5,  
  "eligibleSchemes": 3,  
  "recentApplications": [  
    {  
      "schemeName": "PM Awas Yojana",  
      "status": "Under Review",  
      "appliedDate": "2026-04-01"  
    }  
  ],  
  "notifications": [  
    "Your application has been reviewed",
```

```
"New scheme available"  
]  
}
```

Eligibility Check Form Module

The Eligibility Check Form is a crucial component of the system that gathers essential user information required to evaluate eligibility for various government welfare schemes. It includes fields such as age, income level, social category, occupation, state, and other relevant criteria. The form is designed to be simple and user-friendly, ensuring that users can input their details accurately without confusion.

Once the user submits the form, the system processes the entered data using predefined rules and conditions associated with each scheme. These rules are based on official eligibility criteria such as income limits, age restrictions, and beneficiary categories. This automated evaluation eliminates the need for manual verification at the initial stage and speeds up the recommendation process.

The form may include validation mechanisms to ensure data accuracy, such as restricting invalid inputs or mandatory field checks. By collecting structured and reliable data, this module plays a key role in delivering accurate eligibility results and guiding users toward appropriate schemes.

Sample Scheme Data (JSON)

```
{  
  "userId": "U101",  
  "age": 24,  
  "income": 250000,  
  "category": "OBC",
```

```
"occupation": "Student",  
  
"state": "Tamil Nadu",  
  
"disability": false  
}
```

Eligibility Result Module

The Eligibility Result Page presents the outcome of the eligibility analysis performed by the system. It provides users with a clear and categorized list of schemes for which they are eligible and not eligible. This structured presentation helps users easily understand their options and make informed decisions about applying for suitable schemes.

For eligible schemes, the system may display additional details such as benefits, application deadlines, and direct links to apply. For schemes where the user is not eligible, the system may also provide reasons such as exceeding income limits or not meeting age requirements. This transparency helps users understand the criteria and avoid unnecessary applications.

This module enhances user experience by offering actionable insights, such as suggestions to update profile information or explore alternative schemes. By clearly communicating eligibility results, it reduces confusion and ensures that users focus only on relevant opportunities.

Sample Response JSON

```
{  
  
  "userId": "U101",  
  
  "eligibleSchemes": [  
  
    {
```

```
"schemeId": "S101",  
  
"schemeName": "Scholarship Scheme",  
  
"status": "Eligible"  
  
}  
  
],  
  
"notEligibleSchemes": [  
  
  {  
  
    "schemeId": "S102",  
  
    "schemeName": "Farmer Subsidy",  
  
    "reason": "Income too high"  
  
  }  
  
]  
  
}
```

Track Applications Module

The Track Applications Page allows users to monitor the progress of their submitted applications in a transparent and structured manner. It provides real-time updates on application statuses such as Pending, Under Review, Approved, or Rejected. This helps users stay informed about the progress without needing to contact authorities manually.

The module also displays additional details such as application submission date, last updated time, and any remarks provided by reviewing officers. These details offer better clarity on the current stage of the application and any required actions from the user. In some cases, notifications or alerts may be triggered when there is a status change.

The Track Applications Page improves system transparency and user trust by ensuring that users have complete visibility over their application lifecycle. It reduces uncertainty and enhances communication between users and the system, making the entire process more efficient and user-friendly.

Sample JSON Format

```
{  
  "userId": "U101",  
  "applications": [  
    {  
      "applicationId": "A501",  
      "schemeName": "Startup India",  
      "status": "Approved",  
      "lastUpdated": "2026-04-05"  
    }  
  ]  
}
```

Officer Review Dashboard Module

The Officer Review Dashboard is designed for authorized officers responsible for verifying and processing user applications. It provides a structured interface where officers can view application details, user information, and uploaded documents. This centralized system ensures efficient handling of applications without relying on manual processes.

Officers can review submitted data, verify documents, and update application statuses such as Approved, Rejected, or Under Review. The

dashboard may also include filters and search options to help officers quickly find specific applications based on criteria like date, scheme, or status. This improves productivity and reduces processing time.

This module ensures accountability and accuracy in the decision-making process. By maintaining proper records of actions taken by officers, it supports transparency and helps in auditing if required. Overall, it plays a vital role in ensuring that only eligible applications are approved.

Sample JSON Format

```
{  
  "officerId": "O201",  
  "pendingApplications": [  
    {  
      "applicationId": "A501",  
      "userName": "Karthi",  
      "schemeName": "Startup India",  
      "documentsVerified": false  
    }  
  ]  
}
```

Admin Applications Management Module

The Admin Applications Management module provides administrators with complete control over all applications within the system. It allows admins to view, manage, and organize application data efficiently. This module ensures that

the system operates smoothly and that all application records are maintained accurately.

Administrators can perform actions such as updating application details, resolving inconsistencies, and removing duplicate or invalid entries. Unlike officers, admins have higher-level permissions, enabling them to oversee the entire system rather than focusing on individual applications.

This module plays a key role in maintaining data integrity and system performance. By monitoring application trends and managing records effectively, administrators can ensure that the platform remains organized and efficient. It also supports decision-making by providing a complete overview of system activities.

Sample JSON Format

```
{  
  "adminId": "ADM1",  
  "totalApplications": 120,  
  "applications": [  
    {  
      "applicationId": "A501",  
      "userId": "U101",  
      "schemeName": "Startup India",  
      "status": "Approved"  
    }  
  ]  
}
```

System Administration Dashboard Module

The System Administration Dashboard provides a comprehensive overview of the entire platform. It enables administrators to monitor key system metrics such as total users, number of schemes, and active applications. This centralized view helps in understanding overall system performance and usage.

The dashboard also includes functionalities for managing users and schemes. Administrators can add new schemes, update existing ones, and manage user accounts. This ensures that the platform remains up-to-date with the latest government initiatives and user data.

The module may include reporting and analytics features that provide insights into system usage patterns. These insights help administrators make informed decisions and improve system efficiency. Overall, this dashboard is essential for maintaining smooth and effective system operations.

Sample JSON Format

```
{  
  "adminId": "ADM1",  
  "totalUsers": 500,  
  "totalSchemes": 100,  
  "activeApplications": 300,  
  "systemLogs": [  
    "User U101 logged in",  
    "New scheme added"  
  ]  
}
```


Officer Dashboard Module

The Officer Dashboard is a simplified interface designed to help officers manage their daily tasks efficiently. It provides an overview of assigned applications, completed reviews, and pending tasks. This helps officers prioritize their work and manage time effectively.

The dashboard displays key metrics such as the number of applications assigned, processed, and awaiting action. It may also include alerts or reminders for pending reviews to ensure timely processing. This structured approach improves workflow efficiency and reduces delays.

The Officer Dashboard enhances productivity by offering quick access to essential tools and information. By streamlining task management and providing clear insights into workload, it ensures that officers can perform their responsibilities effectively and contribute to faster application processing.

Sample JSON Format

```
{  
  "officerId": "O201",  
  "name": "Officer Kumar",  
  "assignedApplications": 25,  
  "reviewedApplications": 10,  
  "pendingReviews": 15  
}
```

4.3 Database Design

Database integration in the Welfare Advisory System plays a crucial role in managing and storing structured data such as user details, scheme information, and eligibility criteria. The system uses MySQL as the relational database and integrates it with the backend using Spring Data JPA (Hibernate). This integration allows seamless communication between the application and the database.

The database is designed to ensure data consistency, efficient retrieval, and scalability. All data operations such as insertion, retrieval, updating, and deletion are handled through the backend, ensuring controlled and secure access to the database.

Working of Database Integration

When a user interacts with the system, the backend processes the request and performs the required database operations. For example, when scheme recommendations are generated, the system retrieves scheme data from the database and filters it based on user eligibility.

The integration ensures that:

- Data is stored in a structured format
- Queries are executed efficiently
- Relationships between tables are maintained
- Data can be accessed quickly

Entity Mapping (Database to Java Objects)

The system uses Entity classes to map database tables to Java objects. Each entity represents a table, and each attribute corresponds to a column in that table.

Repository Interface (Database Operations)

The system uses repository interfaces to perform database operations without writing SQL queries.

Example: Scheme Repository

```
public interface SchemeRepository extends JpaRepository<Scheme, Long> {  
    List<Scheme> findByCategory(String category);  
    List<Scheme> findByMaxIncomeGreaterThanOrEqualTo(double income);  
}
```

This interface provides built-in methods such as `save()`, `findAll()`, and `delete()`. Custom methods like `findByCategory()` allow fetching data based on specific conditions. This reduces complexity and improves development speed.

Database Configuration

The connection between the backend and the database is configured in the application properties file.

```
spring.datasource.url=jdbc:mysql://localhost:3306/welfare_db  
spring.datasource.username=root  
spring.datasource.password=yourpassword  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

Explanation

- `spring.datasource.url` → Specifies database location
- `username` & `password` → Authentication credentials
- `ddl-auto=update` → Automatically updates table structure

- `show-sql=true` → Displays SQL queries in console

Data Retrieval and Processing

When the backend receives a request, it retrieves data from the database and processes it based on business logic.

Example Data Fetching

```
List<Scheme> schemes = schemeRepository.findAll();
```

This line retrieves all scheme records from the database. The data is then filtered based on user eligibility conditions before being sent to the frontend.

Relationship Handling

The database maintains relationships between different entities such as users and schemes. These relationships are managed using primary and foreign keys.

For example:

- One user can be associated with multiple schemes
- A scheme can apply to multiple users

This relational structure ensures efficient data organization and retrieval.

Users Table

The Users table stores all registered user information in the system. It includes personal details such as name, email, age, income, and category, which are used to check eligibility for schemes. Each user has a unique ID that is used to connect with other tables like applications and eligibility results.

Table No 4.1 User table plays a key role in maintaining user data and personalization.

Field Name	Data Type	Description
user_id	INT (PK)	Unique user ID
name	VARCHAR(100)	User name
email	VARCHAR(100)	User email
age	INT	User age
income	DECIMAL	User income
category	VARCHAR(50)	Social category

Schemes Table

The Schemes table contains details of all government welfare schemes available in the system. It includes eligibility criteria such as age and income limits, along with benefits provided by each scheme. This table is used to match user data with scheme requirements during eligibility checking.

Table No 4.2 Scheme Table helps the system recommend suitable schemes to users.

Field Name	Data Type	Description
scheme_id	INT (PK)	Unique scheme ID
scheme_name	VARCHAR(150)	Scheme name
min_age	INT	Minimum age
max_age	INT	Maximum age
max_income	DECIMAL	Income limit
benefits	TEXT	Scheme benefits

Applications Table

The Applications table stores all scheme applications submitted by users. It keeps track of which user applied for which scheme and the current status of the application.

Table No 4.3 Application table is important for tracking progress and managing approvals or rejections. It is also used in both user and officer dashboards.

Field Name	Data Type	Description
application_id	INT (PK)	Unique application ID
user_id	INT (FK)	Reference to user
scheme_id	INT (FK)	Reference to scheme
status	VARCHAR(50)	Application status
applied_date	DATE	Date of application

Eligibility Result Table

The Eligibility Result table stores the results of the eligibility check process. It records whether a user is eligible or not for a particular scheme. If the user is not eligible, the system may also store the reason.

Table No 4.4 Eligibility Result Table helps users understand why they qualify or do not qualify for certain schemes.

Field Name	Data Type	Description
result_id	INT (PK)	Unique result ID
user_id	INT (FK)	Reference to user
scheme_id	INT (FK)	Reference to scheme
status	VARCHAR(20)	Eligible or Not Eligible
reason	TEXT	Reason if not eligible

Officer Table

The Officer table stores information about government officers who are responsible for reviewing user applications. It includes details such as name, email, and department, which help in identifying and managing officers within the system. Each officer is assigned a unique ID to track their actions.

Table No 4.5 Officer Table is connected to the application review process, where officers verify documents and update application statuses.

Field Name	Data Type	Description
officer_id	INT (PK)	Unique officer ID
name	VARCHAR(100)	Officer name
email	VARCHAR(100)	Officer email
password	VARCHAR(255)	Encrypted password
department	VARCHAR(100)	Officer department

CHAPTER 5

IMPLEMENTATION AND EXPERIMENTATION RESULT

5.1 Implementation

This chapter presents the implementation details of the Welfare Advisory System along with the output obtained from different modules. The system has been successfully developed by integrating frontend, backend, and machine learning components to provide accurate and efficient recommendations of government welfare schemes. Each module has been implemented with a focus on functionality, performance, and ease of use.

The implementation emphasizes creating a user-friendly interface that allows users to interact with the system easily without requiring technical knowledge. It also incorporates secure authentication mechanisms to ensure safe access to user data and role-based functionalities.

The integration of machine learning enhances the system's ability to provide intelligent and personalized recommendations based on user input.

The system has been tested across various modules to ensure proper functionality and reliability in real-time scenarios. The following sections present the outputs of different modules along with their explanations, demonstrating how the system processes user input, generates results, and supports decision-making effectively.

Fig No 5.1 Shows the welfare advisory systems Implementation Diagram

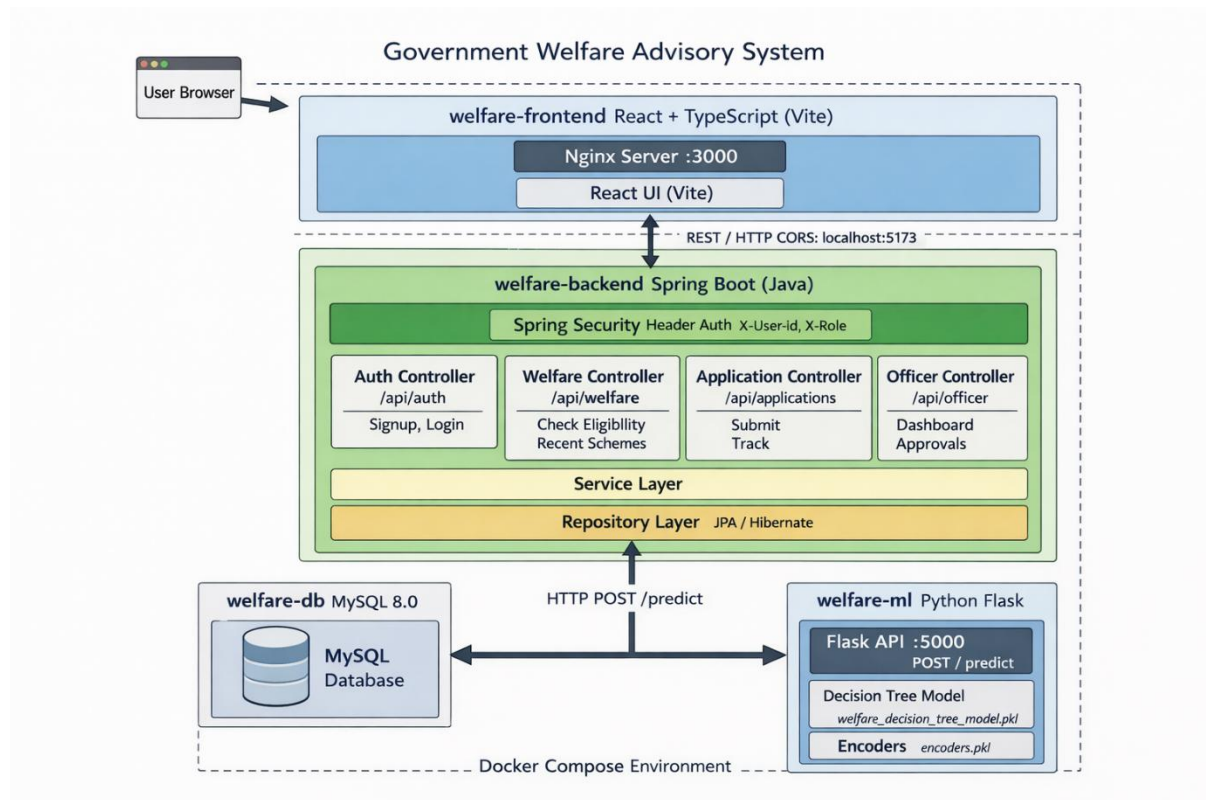


Fig No 5.1 Implementation Diagram

1. Presentation Layer (Frontend Layer)

The presentation layer is responsible for all user interactions and acts as the interface between the user and the system. It is developed using modern web technologies such as HTML, CSS, and JavaScript (or React for advanced implementation). This layer ensures that users can easily input their data and view the results in a clear and structured manner.

The frontend consists of multiple components such as:

- User input forms for entering personal and socio-economic details
- Dashboard interface for displaying recommended schemes
- Navigation components for easy movement across pages

This layer is designed with a strong focus on usability and accessibility, ensuring that users from different backgrounds, including non-technical users, can interact with the system without difficulty. Form validation is implemented at this level to ensure that incorrect or incomplete data is not sent to the backend.

The frontend uses asynchronous communication (AJAX or Fetch API) to interact with the backend without reloading the page. This improves user experience by providing faster responses and smooth interactions.

2. Application Layer (Backend Layer)

The application layer serves as the core processing unit of the system. It is implemented using the Spring Boot framework, which provides a robust environment for building scalable and secure web applications.

This layer is responsible for:

- Processing user inputs received from the frontend
- Implementing business logic
- Communicating with the machine learning model
- Fetching and filtering data from the database
- Sending processed results back to the frontend

The backend follows a layered architecture pattern, which includes:

a) Controller Layer

Handles incoming HTTP requests and routes them to appropriate services. It acts as the entry point of the application.

b) Service Layer

Contains the core business logic. It processes user input, interacts with external services (ML API), and applies filtering rules.

c) Repository Layer

Handles database interactions using JPA/Hibernate. It performs operations such as data retrieval, insertion, and updates.

This separation of responsibilities ensures that the system is well-organized and easy to maintain.

3. Data Layer (Database + ML Model)

The data layer consists of two major components:

a) Database (MySQL)

The database stores structured data related to users, government schemes, eligibility criteria, and application records. It is designed using a relational model to ensure data consistency and efficient querying.

b) Machine Learning Model (Flask API)

The machine learning component plays a crucial role in determining user eligibility. It is implemented using Python and deployed as a Flask API. The model takes user inputs and calculates an eligibility score based on trained data.

The separation of the ML model into an independent service allows flexibility, as the model can be updated or improved without affecting the backend system.

Inter-Layer Communication

Communication between different layers is achieved using RESTful APIs, which follow standard HTTP protocols. The frontend sends requests to the backend in JSON format, and the backend processes these requests and returns responses accordingly.

The backend also communicates with the machine learning API using REST calls. This ensures that the ML model remains loosely coupled with the main application.

Data Flow Explanation

1. The user enters data in the frontend form
2. The frontend sends a POST request to the backend API
3. The backend validates and processes the data
4. The backend sends the data to the ML API
5. The ML model returns an eligibility score
6. The backend filters schemes based on this score
7. The final result is sent back to the frontend

This structured flow ensures efficient processing and accurate results.

Frontend Technologies

- React.js – Used to build an interactive and responsive user interface.
- TypeScript – Provides type safety and improves the reliability of frontend code.
- Tailwind CSS – Used for styling and creating a modern, responsive design.
- Vite – A fast development tool used for building and running the frontend application.

Backend Technologies

- Spring Boot – Used to develop the backend REST APIs and manage application logic.
- Java – Primary programming language used for backend development.

- Spring Data JPA – Used to simplify database operations and manage entity relationships.

Database Technology

- MySQL – Used to store structured data such as users, schemes, applications, and eligibility records.

Machine Learning Technologies

- Python – Used to develop the machine learning model.
- Scikit-learn – Machine learning library
- Flask/FastAPI – Used to expose the machine learning model as an API

Development Tools

- Git – Used for version control and project collaboration.
- Postman – Used for testing backend api endpoints.

5.2 Experimentation Result

5.2.1 User Dashboard

The user dashboard serves as the central interface where users can access all functionalities of the system. It provides a summary of available actions and allows users to navigate easily between different modules such as eligibility checking and scheme browsing.

The dashboard is designed to be interactive and user-friendly, ensuring that users can quickly access important features without unnecessary complexity. It improves overall user experience by organizing functionalities in a structured manner.

<h2>Welcome User</h2>

[Check Eligibility](/check)

[Track Applications](/track)

[Dashboard](/dashboard)

[Schemes](/schemes)

[Help](/help)

Figure Description 5.2.1 The figure shows the user dashboard displaying available features and quick access options.

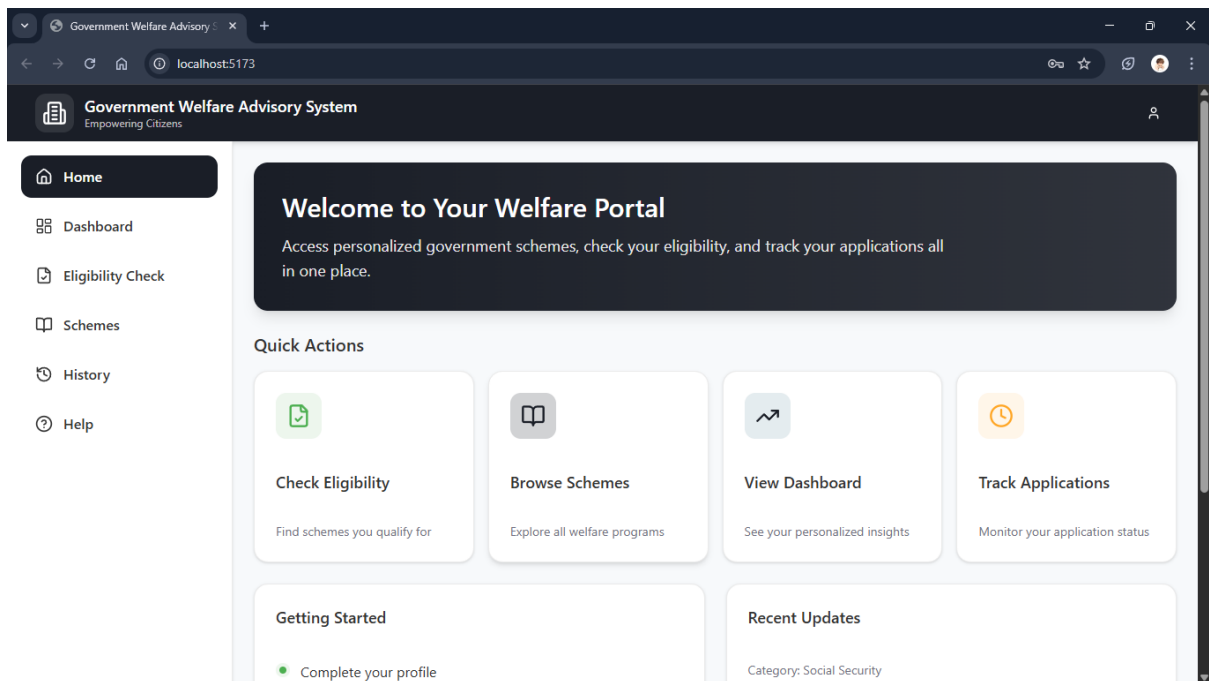


Fig No 5.2.1 User Dashboard

The dashboard provides users with quick access to important features such as eligibility checking, scheme browsing, and application tracking. These features are presented in the form of cards, making navigation simple and intuitive.

It dynamically displays content based on the user's activity, ensuring a personalized experience. This helps users easily find relevant information and perform actions efficiently.

The design also ensures consistency and responsiveness, allowing users to access the dashboard from different devices without any issues.

5.2.2 Eligibility Check Form

The eligibility check form is used to collect user details required for determining suitable schemes. It is designed in a step-by-step format, making it easy for users to provide necessary information without confusion.

The form ensures proper validation of input data, preventing errors and improving accuracy. This module plays a key role in collecting reliable data for the recommendation process.

```
<form id="eligibilityForm">  
  
  <input type="number" id="age" placeholder="Age">  
  
  <input type="number" id="income" placeholder="Income">  
  
  <input type="text" id="occupation" placeholder="Occupation">  
  
  <button type="submit">Check Eligibility</button>  
  
</form>
```

```
fetch("/predict", {  
  
  method: "POST",  
  
  headers: {"Content-Type": "application/json"},  
  
  body: JSON.stringify(userData)  
  
});
```

Figure Description 5.2.2 The figure shows the eligibility form where users enter personal details.

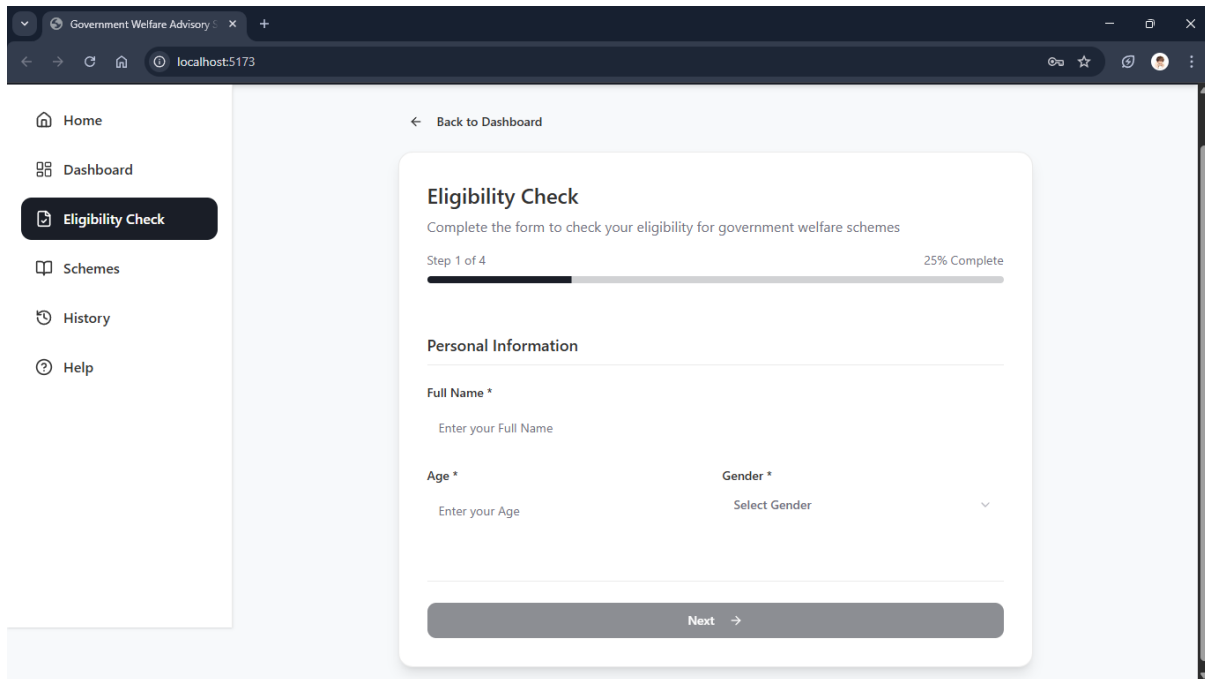
The image is a screenshot of a web browser displaying a web application. The browser's address bar shows 'localhost:5173'. On the left, there is a sidebar menu with links: 'Home', 'Dashboard', 'Eligibility Check' (which is highlighted with a dark background), 'Schemes', 'History', and 'Help'. The main content area has a 'Back to Dashboard' link at the top left. The central focus is the 'Eligibility Check' form. The form has a title 'Eligibility Check' and a subtitle 'Complete the form to check your eligibility for government welfare schemes'. Below this is a progress indicator showing 'Step 1 of 4' and '25% Complete' with a corresponding progress bar. The form section is titled 'Personal Information'. It contains three input fields: 'Full Name *' with a placeholder 'Enter your Full Name', 'Age *' with a placeholder 'Enter your Age', and 'Gender *' with a placeholder 'Select Gender' and a dropdown arrow. At the bottom of the form is a dark grey button labeled 'Next' with a right-pointing arrow.

Fig No 5.2.2 Eligibility Check Form

The eligibility form collects essential user data such as name, age, and gender. This information is required to determine eligibility for various schemes. The step-by-step design ensures that users can complete the process easily.

Once the form is submitted, the data is sent to the backend for processing. The backend then interacts with the machine learning model to calculate an eligibility score.

This module ensures accurate data collection and smooth interaction between the user and the system, improving the overall efficiency of the recommendation process.

5.2.3 Eligibility Result Page

The eligibility result page displays the schemes that match the user's profile. It provides a clear and structured output, allowing users to easily understand the results.

The page ensures that only relevant schemes are shown, reducing confusion and improving user satisfaction.

Backend – ML Call

```
public double getEligibilityScore(User user) {  
  
    RestTemplate restTemplate = new RestTemplate();  
  
    String url = "http://localhost:5000/predict";  
  
    ResponseEntity<Double> response =  
  
        restTemplate.postForEntity(url, user, Double.class);  
  
    return response.getBody();  
  
}
```

Controller

```
@PostMapping("/predict")  
  
public List<Scheme> predict(@RequestBody User user) {  
  
    double score = mlService.getEligibilityScore(user);  
  
    return schemeService.getEligibleSchemes(user);  
  
}
```

Python ML API

```
from flask import Flask, request, jsonify
```

```

import pickle

app = Flask(__name__)

model = pickle.load(open("model.pkl", "rb"))

@app.route('/predict', methods=['POST'])
def predict():

    data = request.get_json()

    features = [data['age'], data['income']]

    prediction = model.predict([features])[0]

    return jsonify(prediction)

app.run()

```

Figure Description 5.2.3 The figure shows the result page displaying eligible schemes.

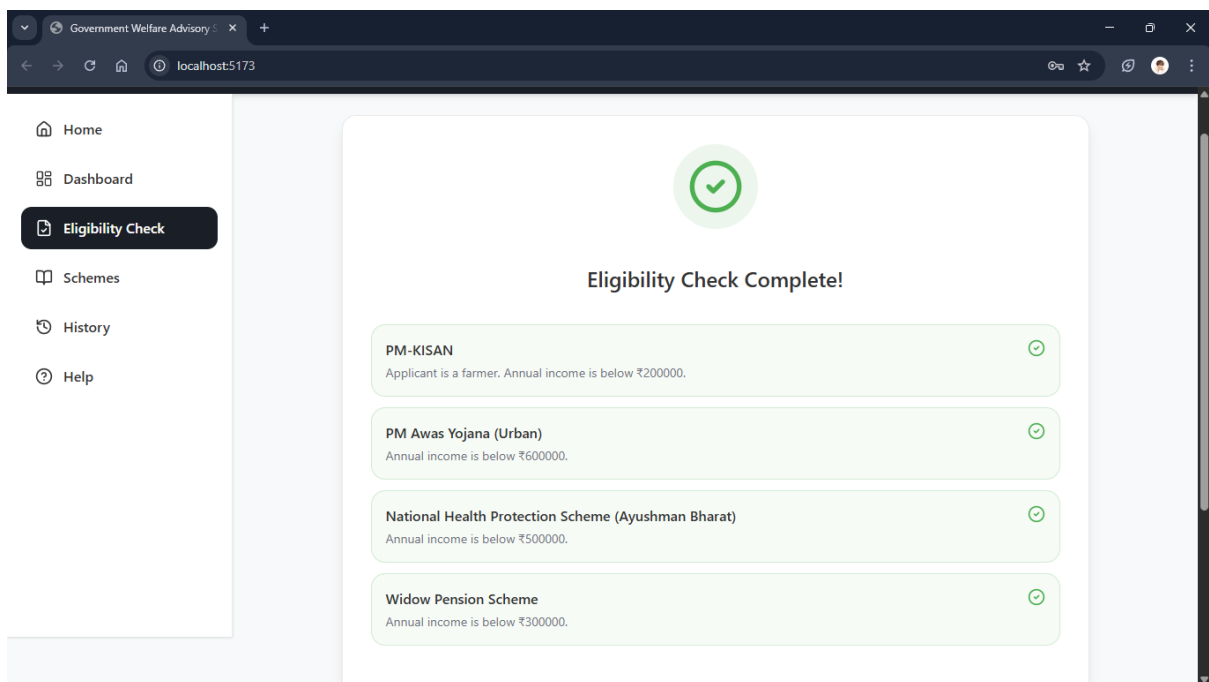


Fig No 5.2.3 Eligibility Result Page

The result page presents the recommended schemes based on user input. Each scheme is displayed with relevant details, helping users understand their eligibility clearly.

The recommendations are generated using a combination of machine learning predictions and rule-based filtering.

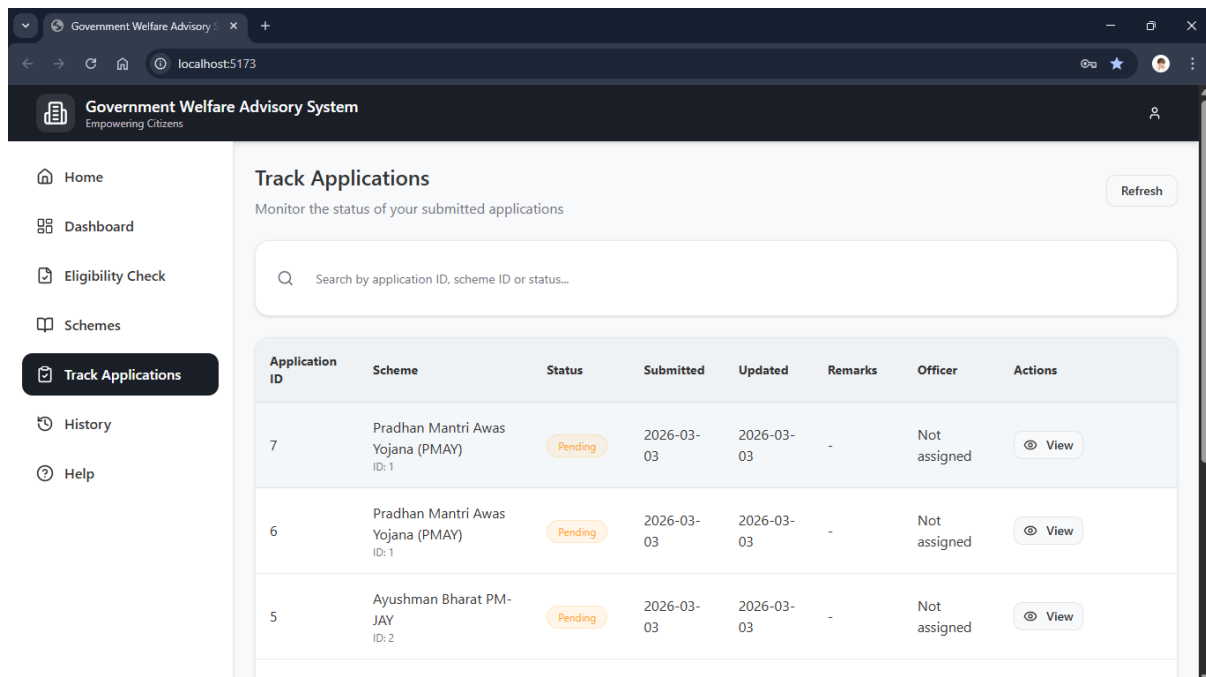
The layout is designed to be simple and readable, allowing users to quickly identify suitable schemes and proceed with further actions if required.

5.2.4 Track Applications Page

The track applications page allows users to monitor the status of their submitted applications.

This module improves transparency and helps users stay informed about their application progress.

Figure Description 5.2.4 The figure shows the application tracking page with status details.



Application ID	Scheme	Status	Submitted	Updated	Remarks	Officer	Actions
7	Pradhan Mantri Awas Yojana (PMAY) ID: 1	Pending	2026-03-03	2026-03-03	-	Not assigned	View
6	Pradhan Mantri Awas Yojana (PMAY) ID: 1	Pending	2026-03-03	2026-03-03	-	Not assigned	View
5	Ayushman Bharat PM-JAY ID: 2	Pending	2026-03-03	2026-03-03	-	Not assigned	View

Fig No 5.2.4 Track Applications Page

The tracking module enables users to view the progress of their applications in real time. It displays information in a tabular format, making it easy to understand.

```
<table>

<tr><th>Scheme</th><th>Status</th></tr>

<tr><td>PMAY</td><td>Approved</td></tr>

</table>
```

Users can search and filter applications based on different criteria, improving usability and efficiency.

This feature enhances user experience by providing clear updates and ensuring that users remain informed about their application status.

5.2.5 Officer Review Dashboard

The officer dashboard provides administrative users with an overview of application statistics and system activity. It helps officers manage applications efficiently.

The dashboard displays summarized data, making it easier to analyze and process applications.

```
@GetMapping("/applications")

public List<Application> getAllApplications() {

    return applicationRepository.findAll();

}
```

Figure Description 5.2.5 The figure shows the officer dashboard displaying application statistics.

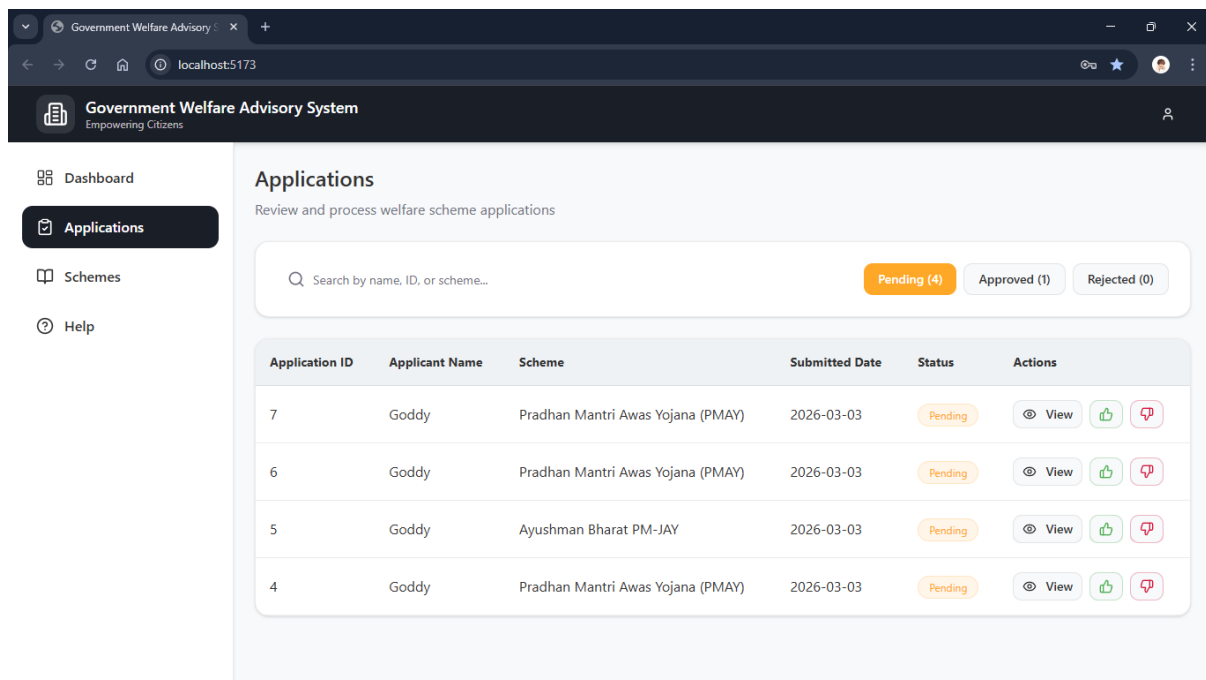


Fig No 5.2.5 Officer Review Dashboard

The officer dashboard provides insights into system activity, including pending, approved, and rejected applications. This helps officers monitor system performance.

It also displays recent applications, allowing quick access to newly submitted requests.

The module improves efficiency by providing tools for managing applications and ensuring smooth workflow.

5.2.6 Admin Applications Management

The admin module allows administrators to manage applications and system data. It ensures proper control over application processing.

The module provides options to approve, reject, or assign applications, improving workflow management.

```

@GetMapping("/admin/applications")

public List<Application> getAllApplications() {

    return applicationRepository.findAll();

}

@DeleteMapping("/admin/delete/{id}")

public ResponseEntity<String> deleteApplication(@PathVariable Long
id) {

    applicationRepository.deleteById(id);

    return ResponseEntity.ok("Application Deleted Successfully");

}

@GetMapping("/admin/application/{id}")

public ResponseEntity<Application> getApplication(@PathVariable Long
id) {

    return applicationRepository.findById(id)

        .map(ResponseEntity::ok)

        .orElse(ResponseEntity.notFound().build());

}

```

Figure Description 5.2.6 The figure shows the admin interface for managing applications.

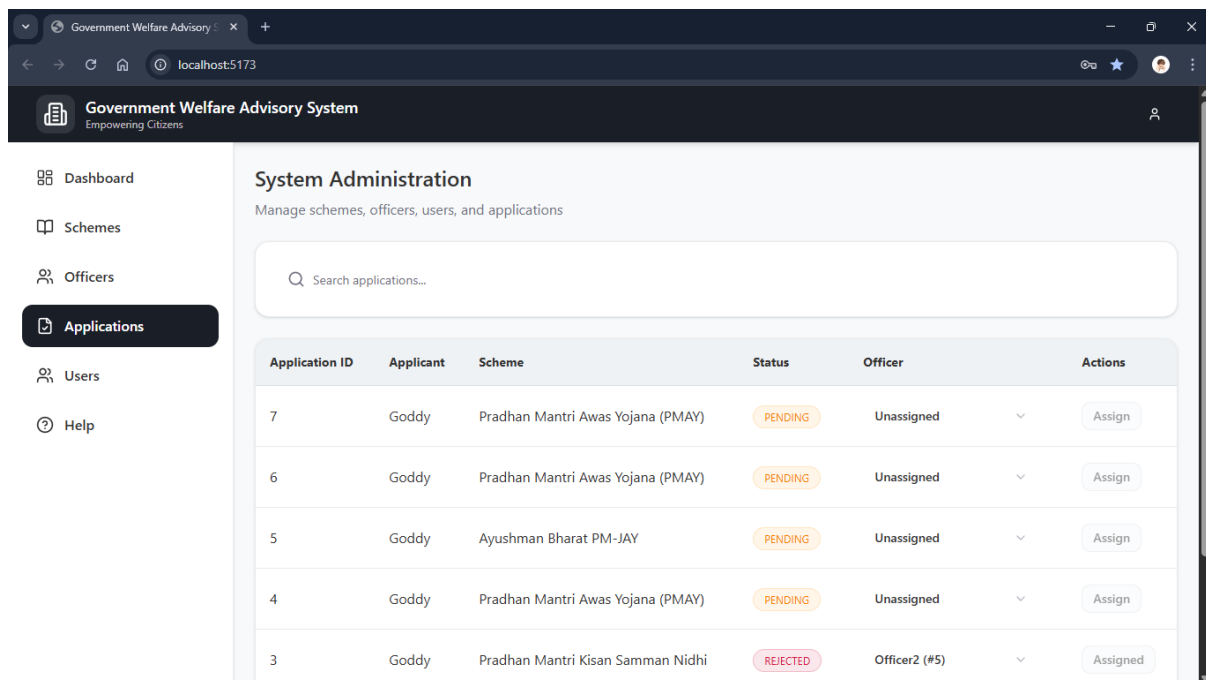


Fig No 5.2.6 Admin Applications Management

The admin interface displays all applications along with relevant details such as applicant name, scheme, and status. This ensures transparency in the system.

Administrators can perform actions such as approving or rejecting applications based on eligibility criteria.

This module ensures efficient system management and plays a key role in maintaining the overall functionality of the system.

5.2.7 System Administration Dashboard

The system administration dashboard provides a centralized interface for managing all system-level operations. It is designed for administrators to monitor overall system performance, manage users, schemes, officers, and applications efficiently. The dashboard presents key statistics and summaries that help in quick decision-making.

This module ensures that administrators have complete control over the system. It organizes information in a structured format, allowing easy navigation between different management sections such as users, applications, and schemes.

```
@PostMapping("/admin/add-scheme")
```

```
public Scheme addScheme(@RequestBody Scheme scheme) {  
    return schemeRepository.save(scheme);  
}
```

```
@GetMapping("/admin/schemes")
```

```
public List<Scheme> getAllSchemes() {  
    return schemeRepository.findAll();  
}
```

```
@DeleteMapping("/admin/delete-scheme/{id}")
```

```
public ResponseEntity<String> deleteScheme(@PathVariable Long id) {  
    schemeRepository.deleteById(id);  
    return ResponseEntity.ok("Scheme Deleted Successfully");  
}
```


Figure Description 5.2.7 The figure shows the system administration dashboard displaying overall statistics such as total users, schemes, applications, and officers.

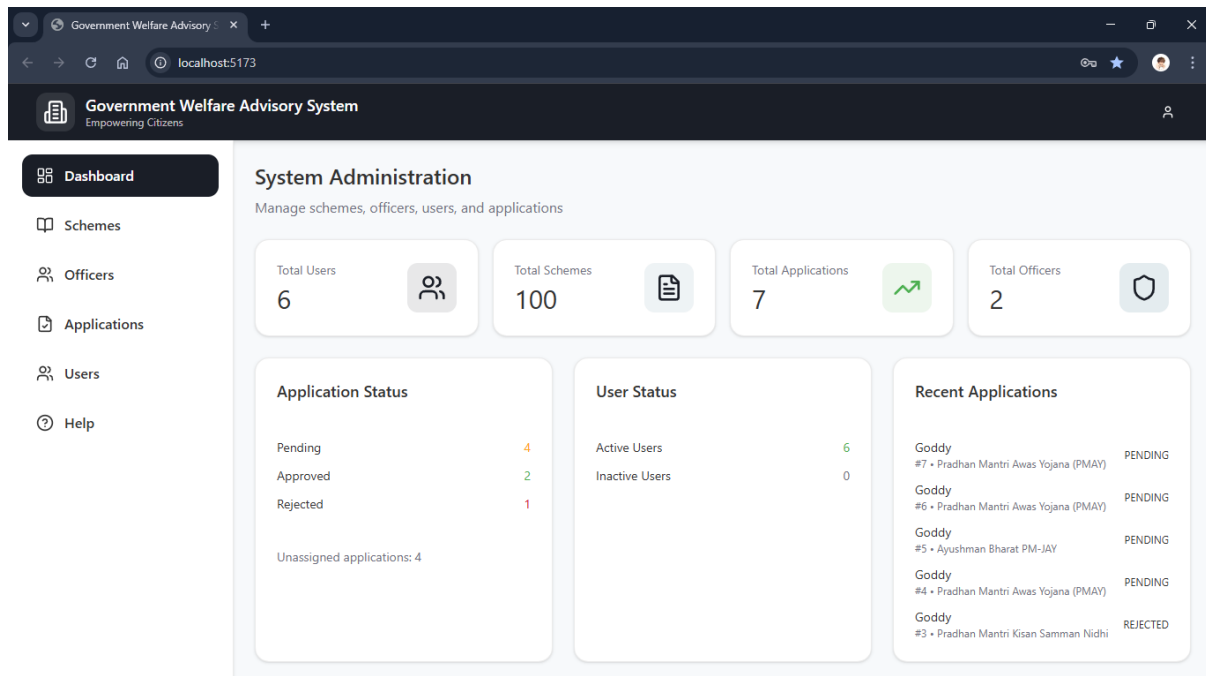


Fig No 5.2.7 System Administration Dashboard

The administration dashboard provides a summary view of important system metrics, including total users, total schemes, total applications, and total officers. These metrics help administrators quickly understand the current state of the system and monitor its usage.

It also includes sections such as application status and user status, which display categorized data like pending, approved, and rejected applications, as well as active and inactive users. This helps in identifying system trends and managing workload effectively.

The dashboard displays recent applications, allowing administrators to track newly submitted requests. This feature improves response time and ensures that applications are processed without delay, enhancing overall system efficiency.

5.2.8 Officer Dashboard

The officer dashboard is designed to assist officers in managing and processing user applications. It provides a focused view of application-related data, enabling officers to perform their tasks efficiently. The dashboard presents key statistics and recent activities to simplify decision-making.

This module plays an important role in the workflow of the system by bridging the gap between users and administrators. It ensures that applications are reviewed, processed, and updated in a timely manner, maintaining system reliability.

```
<h2>Officer Dashboard</h2>

<div id="applications"></div>

<script>

fetch("/officer/applications")

.then(res => res.json())

.then(data => {

  let container = document.getElementById("applications");

  data.forEach(app => {

    let div = document.createElement("div");

    div.innerHTML = app.schemeName + " - " + app.status;

    container.appendChild(div);

  });

});

</script>
```

```

@PutMapping("/officer/update-status/{id}")

public ResponseEntity<String> updateStatus(@PathVariable Long id,

                                           @RequestParam String status) {

    Application app = applicationRepository.findById(id).orElse(null);

    if (app != null) {

        app.setStatus(status);

        applicationRepository.save(app);

        return ResponseEntity.ok("Updated");

    }

    return ResponseEntity.notFound().build();

}

```

Figure Description 5.2.8 The figure shows the officer dashboard displaying application statistics and recently submitted applications.

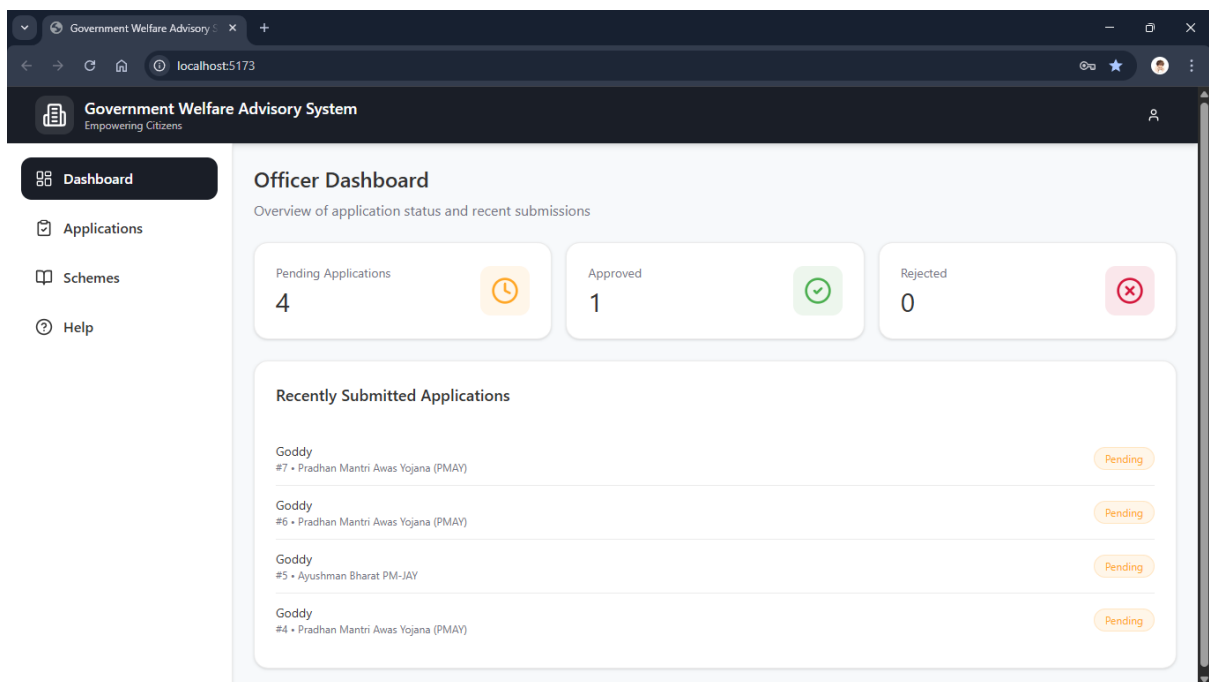


Fig No 5.2.8 Officer Dashboard

The officer dashboard displays key metrics such as the number of pending, approved, and rejected applications. These statistics help officers understand their workload and prioritize tasks accordingly. It provides a clear overview of application processing status.

The module also includes a section for recently submitted applications, allowing officers to quickly access and review new requests. This ensures that no application is overlooked and improves processing efficiency.

The dashboard enhances usability by presenting information in a clean and organized layout. It supports efficient workflow management, enabling officers to handle applications systematically and ensure timely updates.

CHAPTER 6

CONCLUSION

The Government Welfare Advisory System was developed to simplify the process of discovering and accessing government welfare schemes. Many citizens face difficulties in identifying schemes they are eligible for due to scattered information and complex eligibility criteria. This project addresses these challenges by providing a centralized digital platform where users can easily check their eligibility, explore suitable schemes, apply for benefits, and track the status of their applications.

The system integrates modern technologies such as React for frontend development, Spring Boot for backend services, MySQL for database management, and machine learning for eligibility prediction. By analyzing user information such as age, income, occupation, and social category, the system provides personalized recommendations for welfare schemes. This reduces the time and effort required for manual searching and helps citizens identify relevant programs more efficiently.

The implementation of role-based access control allows different users such as citizens, officers, and administrators to interact with the system according to their responsibilities. Citizens can apply for schemes and monitor their application progress, while officers can review and process applications. Administrators manage scheme information and system users, ensuring smooth operation of the platform.

The experimental results show that the system successfully improves the process of scheme discovery and application management. The machine learning model effectively predicts eligibility and assists users in finding suitable welfare programs. The integration of web technologies and intelligent data analysis helps create a more accessible and transparent system for welfare scheme distribution.

In conclusion, the Government Welfare Advisory System demonstrates how technology can be used to improve public service delivery and increase awareness of government welfare programs. The system provides a user-friendly, efficient, and scalable solution that helps connect citizens with the benefits intended for them.

6.2 Future Enhancements

- Integration of advanced machine learning models to improve the accuracy of eligibility prediction and welfare scheme recommendations.
- Development of a mobile application to make the platform more accessible for users who prefer accessing services through smartphones.
- Implementation of multilingual support to allow citizens from different regions to use the system in their preferred language.
- Addition of real-time notifications and alerts to inform users about new schemes, application deadlines, and status updates.
- Integration with government databases to automate verification of user information and reduce manual processing.
- Introduction of advanced search features that allow users to find schemes using simple keywords or natural language queries.
- Implementation of data analytics dashboards for administrators to analyze scheme usage, application trends, and beneficiary statistics.
- Enhancement of security features and authentication mechanisms to protect user data and ensure safe access to the platform.

APPENDIX

A1. Source Code

1.ml_service

```
from flask import Flask, request, jsonify

import joblib

import pandas as pd

app = Flask(__name__)

model = joblib.load("../data/processed/welfare_decision_tree_model.pkl")

encoders = joblib.load("../data/processed/encoders.pkl")

FEATURE_ORDER = [

    "age", "gender", "category", "state", "occupation", "annual_income", "disability",

    "is_minority"

]

def safe_label_encode(series, encoder):

    series = series.astype(str)

    return series.apply(

        lambda x: encoder.transform(["Unknown"])[0]

        if x not in encoder.classes_

        else encoder.transform([x])[0]

    )
```

```

@app.route("/predict", methods=["POST"])

def predict_score():

    data = request.json

    df = pd.DataFrame([data])

    for col in FEATURE_ORDER:

        if col not in df:

            df[col] = "Unknown"

    for col, encoder in encoders.items():

        if col in df:

            df[col] = safe_label_encode(df[col], encoder)

    df = df.loc[:, FEATURE_ORDER]

    score = model.predict_proba(df)[0][1]

    return jsonify({

        "eligibility_score": float(score)

    })

if __name__ == "__main__":

    app.run(port=5000, debug=True)

```

2.Scheme Recommendation Service

```

package com.project.welfare.service;

import java.util.ArrayList;

import java.util.List;

```



```

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

import com.project.welfare.dto.*;

@Service

public class SchemeRecommendationService {

    @Autowired

    private SchemeRepository schemeRepository;

    public List<SchemeRecommendationDto> recommendSchemesWithReasons(

        WelfareRequestDto user,

        String eligibilityStatus) {

        if (eligibilityStatus.equals("NOT_ELIGIBLE")) {

            return List.of();}

        List<SchemeListDto> allSchemes = schemeRepository.findAllForList();

        List<SchemeRecommendationDto> result = new ArrayList<>();

        for (SchemeListDto scheme : allSchemes) {

            Integer minAge = scheme.getMinAge();

            Integer maxAge = scheme.getMaxAge();

            Double maxIncome = scheme.getMaxIncome();

            boolean ageMatch = (minAge == null || minAge <= 0 || user.getAge() >=

minAge);

            if (maxAge != null && maxAge > 0) {

                ageMatch = ageMatch && user.getAge() <= maxAge;}

```

```

        boolean incomeMatch = (maxIncome == null || maxIncome <= 0 ||
user.getAnnual_income() <= maxIncome);

        boolean categoryMatch = matchesField(scheme.getCategory(),
user.getCategory());

        boolean genderMatch = matchesField(scheme.getGender(),
user.getGender());

        boolean locationMatch =

            matchesField(scheme.getLocation(), user.getState()) ||

            isIndiaWide(scheme.getLocation());

        boolean occupationMatch = matchesField(scheme.getOccupation(),
user.getOccupation());

        boolean disabilityMatch =
matchesField(scheme.getDisabilityRequired(), user.getDisability());

        boolean minorityMatch = matchesField(scheme.getMinorityRequired(),
user.getIs_minority());

        if (ageMatch && incomeMatch && categoryMatch && genderMatch
&& locationMatch

            && occupationMatch && disabilityMatch && minorityMatch) {

            StringBuilder reason = new StringBuilder();

            if (minAge != null && minAge >= 60) {

                reason.append("Applicant age is above
").append(minAge).append(". ");

            }

            if ("Farmer".equalsIgnoreCase(scheme.getOccupation())) {

                reason.append("Applicant is a farmer. ");
            }

```

```

        if (maxIncome != null && maxIncome > 0) {
            reason.append("Annual income is below ")
                .append(maxIncome.intValue())
                .append(". ");
        }
        if ("Yes".equalsIgnoreCase(scheme.getDisabilityRequired())) {
            reason.append("Applicant has a disability. ");
        }
        if ("Yes".equalsIgnoreCase(scheme.getMinorityRequired())) {
            reason.append("Applicant belongs to a minority group. ");
        }
        result.add(
            new SchemeRecommendationDto(
                scheme.getId(),
                scheme.getSchemeName(),
                reason.toString().trim()));
    }
    return result;
}

private boolean matchesField(String schemeValue, String userValue) {
    if (schemeValue == null || schemeValue.isBlank()) {
        return true;
    }
    if (userValue == null || userValue.isBlank()) {
        return false;
    }
    String normalizedSchemeValue = schemeValue.trim();
    String normalizedUserValue = userValue.trim();
    return normalizedSchemeValue.equalsIgnoreCase("ANY")

```

```

|| normalizedSchemeValue.equalsIgnoreCase(normalizedUserValue);}

private boolean isIndiaWide(String location) {

    return location != null && location.trim().equalsIgnoreCase("India");}}

```

A2. Github Link

Full Complete Code – <https://github.com/Karthi247/Government-Welfare-Advisory-System.git>

A3. API Endpoints

Table A.1 This table shows the Api Endpoints

API Endpoint	Method	Description
/api/auth/signup	POST	Register a new user
/api/auth/login	POST	User login authentication
/api/welfare/check-eligibility	POST	Check user eligibility for welfare schemes
/api/welfare/history	GET	View eligibility check history
/api/applications	POST	Submit application for a welfare scheme
/api/applications/user/{userId}	GET	Get applications submitted by a user
/api/officer/applications/pending	GET	View pending applications for officer review
/api/officer/application/{id}/approve	PUT	Approve a welfare application
/api/officer/application/{id}/reject	PUT	Reject a welfare application
/api/admin/schemes	GET	Retrieve all welfare schemes

/api/admin/schemes	POST	Add a new welfare scheme
/api/admin/schemes/{id}	PUT	Update scheme information
/api/admin/schemes/{id}	DELETE	Delete a welfare scheme

REFERENCES

[1] M. Reddy, S. Rao, and V. Kumar, “Smart Beneficiary Scheme Mapping System Using Machine Learning,” *International Journal of Creative Research Thoughts (IJCRT)*, 2025.

Link: <https://www.ijcrt.org/papers/IJCRT2504843.pdf>

[2] A. Sharma, R. Kumar, and S. Patel, “GovGenius – Government Schemes Recommendation Web Application,” *ResearchGate*, 2024.

Link: https://www.researchgate.net/publication/385790927_GovGenius_-_Government_Schemes_Recommendation_Web_Application

[3] P. Singh, A. Verma, and K. Gupta, “AI-Driven Welfare Scheme Assistance and Management System,” *International Journal of Research Publication and Reviews (IJRPR)*, vol. 6, no. 4, pp. 1203–1208, 2025.

Link: <https://ijrpr.com/uploads/V6ISSUE4/IJRPR42523.pdf>

[4] React Documentation, “React – A JavaScript Library for Building User Interfaces.”

Link: <https://react.dev/>

[5] Spring Framework, “Spring Boot Reference Documentation.”

Link: <https://spring.io/projects/spring-boot>

[6] T. Mitchell, *Machine Learning*. McGraw-Hill Education, 1997.

Link: <https://www.cs.cmu.edu/~tom/mlbook.html>

[8] Oracle Corporation, “MySQL Documentation.”

Link: <https://dev.mysql.com/doc/>